

Graph Representation Learning for the Armenian Morphological Lexicon

Tatev Stepanyan
American University of Armenia
tatev_stepanyan@edu.aua.am

Nelly Zurabyan[†]
KU Leuven
nelli.zurabyan@kuleuven.be

Abstract—Morphological knowledge is a fundamental component of computational linguistics, yet for many languages, particularly for morphologically rich and low-resource ones, this domain remains digitally siloed. The knowledge is fragmented across unstructured dictionaries, which is inaccessible to automated methods. We present a unified computational framework for the Armenian morphology that integrates the construction of a large-scale machine-readable knowledge graph with downstream graph representation learning. Our pipeline systematically builds a graph of 66,677 lemma nodes and 150,801 typed derivational edges by querying the Armenian Wiktionary MediaWiki API, with all extraction logic rigorously grounded in Dum-Tragut’s reference grammar. This structural foundation embeds essential linguistic metadata directly into the node features. We then apply Graph Neural Network (GNN) architectures, specifically Graph Convolutional Network (GCN), GraphSAGE, Graph Attention Network (GAT), and Relational Graph Convolutional Network (R-GCN) to perform derivational link prediction and derive high-quality node embeddings. Our evaluation, which benchmarks performance against Node2Vec, FastText, and XLM-R via grammatical category clustering (Adjusted Rand Index) and derivational analogy tests, demonstrates that graph-structured learning captures derivational regularities that purely distributional methods miss. The full pipeline, datasets, and embeddings are released as open-source resources to support the computational linguistics and digital Armenian studies communities.

Index Terms—Morphology, Modern Eastern Armenian, Linguistics, Data Construction, Graph Construction

I. INTRODUCTION

The internal structure of a word is rarely incidental. Roots, affixes, and the relations between derived forms carry information that is recoverable without any context. That is the information that distributional models, which learn word representations from co-occurrence patterns, do not explicitly capture. A model trained on large text corpora may learn that two words are semantically related, but it has no principled way to represent why they are related, or how one was formed from the other.

This limitation is sharpest in morphologically rich languages (MRLs), where word meaning and grammatical function are frequently encoded in derivational structure

rather than in fixed word forms. Modern Eastern Armenian (MEA) is a particularly clear case [1]. Its derivational system is extensive and regular – a property that initially made it interesting from an etymological standpoint, and that ultimately made it tractable as a computational problem, once it became clear that the etymological resources needed for a deeper historical analysis did not yet exist in machine-readable form. What does exist is a rich synchronic derivational structure, and that is what this work sets out to model.

The scale of Armenian derivation makes this a non-trivial undertaking. MEA exhibits a multi-layered system of prefixes, suffixes, and stem modifications that generate extensive derivational networks from individual roots. For instance, the word գիր (*gir*, “writing”) produces dozens of derivatives like գրիչ (*gritch*, “pen”), գրութիւն (*grutyun*, “literature”), գրադարան (*gradaran*, “library”). Despite their linguistic importance, these relationships are primarily documented in unstructured lexical resources, making them difficult to integrate into computational systems. To date, no large-scale machine-readable morphological resource exists for Armenian in a form suitable for graph-based learning.

Prior work on morphology-aware Natural Language Processing (NLP) has largely treated word formation as a linear, sequential process. For example, MorphBPE [2], though tested on other MRLs, incorporates morphological boundaries into subword tokenization, while MorphCraft [3], developed for Armenian and Greek specifically, injects the morphological features into transformer-based architectures. However, neither work builds nor uses a derivational graph structure.

Derivational morphology is relational by nature: a single base form connects to multiple derived forms across parts of speech, through multi-step processes that sequential and token-based models cannot straightforwardly represent. Existing morphology-aware approaches encode this structure only implicitly (through subword segmentation or contextual embeddings) neither of which captures typed derivational relations or the hierarchical dependencies between them. For Armenian, where distributional supervision is sparse and standard tokenizers obscure morpheme boundaries, this implicit encoding is particularly unreliable. GNNs offer a more direct fit: lexical items

[†] Supervised the work.

become nodes, derivational relations become typed edges, and the architecture is built to propagate information along exactly the kind of structured, multi-step paths that morphology produces. Yet this approach has not been applied to Armenian.

This work addresses the gap through two contributions: construction and release of a typed derivational morphology graph for MEA, and systematic evaluation of GNN architectures for link prediction and embedding quality on this graph, benchmarked against distributional baselines.

The research is guided by the following questions:

- **RQ1:** To what extent can a high-coverage morphological graph for Armenian be constructed from community-sourced Wiktionary data, and what are the resulting topological properties of such a network?
- **RQ2:** Can GNN-derived node embeddings outperform distributional baselines (FastText, XLM-R, Node2Vec) on grammatical category clustering, measured by Adjusted Rand Index (ARI)?
- **RQ3:** Can a GNN accurately infer held-out derivational links, achieving $\text{Hits@10} \geq 0.35$ and $\text{MRR (Mean Reciprocal Rank)} \geq 0.20$?

The thresholds are set above chance but well below the ceiling of typical knowledge graph benchmarks: Hits@10 exceeds 0.9 on WordNet [4], [5], reflecting the greater sparsity of a derivational graph. Pilot experiments on a 10% graph subset confirmed they are achievable. Hits@10 ≥ 0.35 means the correct derivational partner appears in the top-10 candidates more than one-third of the time, sufficient for semi-automatic lexicographic extension; $\text{MRR} \geq 0.20$ ensures the true target ranks near the top rather than appearing arbitrarily deep in the list.

II. LITERATURE REVIEW

A. Armenian Morphological Resources and Corpus Challenges

The primary reference for MEA word formation is Sergey Galst'yan's School Word-Formation Dictionary [6], which systematically catalogs derivational patterns. Available through the Nayiri platform, it lacks a machine-readable format. Dum-Tragut provides a more formal linguistic descriptive framework [7], whose reference grammar defines the morphological system of MEA, including prefixation, suffixation, compounding, causativization, and detransitivization. We implement these descriptions as a rule-based morphological analyzer to guide the construction of a computational morphology model. For historical lexical origins, Acharyan's Armenian Etymological Dictionary [8] remains the canonical work, but it focuses on diachronic etymology rather than synchronic derivational morphology; it serves as motivation for the broader lack of structured computational resources.

Armenian poses specific tokenization and orthographic challenges for modern NLP. The language uses a unique script (the Armenian alphabet, with two standard orthographies: classical and reformed) alongside consistent

vowel reduction and consonant alternation patterns during derivation. Standard multilingual tokenizers trained on Unicode character distributions treat Armenian as a near-uniform block, missing meaningful morpheme boundaries. Arnett and Bergen [1] show that LLMs perform worse on MRLs, tracing this to subword tokenization that obscures morpheme boundaries.

B. Morphology-Aware Representation Learning

MorphBPE [2] modifies the Byte-Pair Encoding (BPE) algorithm to favor morphologically meaningful segmentations, improving downstream performance on MRLs. However, it fails for derivational hierarchies: the algorithm operates bottom-up over character co-occurrence statistics and has no mechanism to represent directed, typed relations between base and derived forms. The output is a flat token sequence. The semantic class of the suffix, for instance, is entirely opaque to the model.

FastText [9] captures local subword similarity through character n-gram embeddings, approximating morphological relatedness without any explicit graph signal. For Armenian, this gives a partial advantage over pure word-level embeddings: the trigrams of *գրիչ* and *գրութիւն* overlap, signaling shared roots. FastText cannot encode a typed derivational hierarchy. Suffixation becomes geometric proximity – an approximation that conflates nominalization, adjectivalization, and verbalization into a single undifferentiated region.

Transformers such as XLM-R [10] encode contextual semantics but lack explicit lexical graph structure. Derivational relationships between lemmas are implicit in the attention weights, recoverable only when the corpus consistently co-locates base and derived forms in syntactic context – which for Armenian, a low-resource language, it rarely does. The transformer input is a sequence of subword tokens; derivational structure, which is relational and hierarchical, is flattened by sequential encoding. MorphCraft [3] partially addresses this by injecting morphological tag embeddings into Armenian transformer self-attention, but the derivational relations remain implicit and untyped. We include XLM-R as a zero-shot contextual baseline to measure what large-scale pretraining alone achieves on Armenian morphology, without any graph supervision.

C. Graph-Based NLP and Heterogeneous Message Passing

GNNs have been applied to dependency parsing, semantic role labeling, and knowledge graph completion, consistently outperforming sequential models on tasks where relational structure matters [11]. In lexical semantics, resources such as BabelNet [12] demonstrate the value of large-scale semantic graphs for multilingual representation learning. In morphology-aware settings, Botha and Blunsom [13] show that explicitly composing root and affix embeddings improves word representations, while FastText extends this idea via character n-grams.

For heterogeneous graphs, where nodes and edges carry distinct types, standard message passing fails because it treats all neighbors identically. Heterogeneous GNNs condition aggregation on edge and node type, producing relation-specific transformation matrices. R-GCN [4] is the prototypical architecture for this setting: it maintains a separate weight matrix per relation type, decomposed via basis functions to remain tractable when the number of relations is large. Derivational morphology graphs are heterogeneous by design, and ours has 30 relation types. Hu et al. [14] generalize heterogeneous message passing to the Heterogeneous Graph Transformer framework, demonstrating consistent gains over homogeneous aggregation on knowledge graph benchmarks. Whether relational encoding helps depends on edge-type frequency, as we discuss in Section VI-C.

Wiktionary has been used as a multilingual lexical resource before: Zesch et al. [15] extracted semantic relatedness from English Wiktionary, and the DBnary project [16] parsed multiple language editions for cross-lingual lexical networks. To the best of our knowledge, no prior work constructs a grammar-grounded typed derivational graph for Armenian at this scale. Applying GNNs to Armenian morphological resources is likewise unexplored.

For link prediction, the Graph Autoencoder (GAE) framework of Kipf and Welling [17] established dot-product decoding with negative sampling as the standard approach, and it is the training framework adopted here. We evaluate four encoders: GCN [18], GraphSAGE [19], GAT/GATv2 [20], [21], and R-GCN [4] within this framework. Node2Vec [22] is included as a structure-only random-walk baseline.

III. DATASET CONSTRUCTION

A. Data Source and Justification

We use the Armenian Wiktionary because its MediaWiki API allows large-scale automated extraction, its semi-structured markup consistently marks part-of-speech (POS) and derivational relations, and the resource is actively maintained. A key limitation is uneven coverage, because high-frequency lemmas are well-documented, while rare vocabulary may be sparsely annotated.

B. Scraping Pipeline

The pipeline is implemented in Python and queries the MediaWiki API, retrieving raw wikitext. Batching at this scale reduces API calls compared to single-page requests. A 0.5-second inter-batch delay and a retry mechanism with increasing delay ensure API compliance.

C. Seeding and Graph Expansion

The scraper is initialized with two sources of seed words. The first is a manually curated list of common, native Armenian words covering basic semantic categories like kinship terms, body parts, natural phenomena, and high-frequency verbs. The second source is an Hunspell

dictionary for Eastern Armenian, provided by the open-source Vortan project [23], contributing several thousand additional lemmas to the initial queue. Proper nouns are excluded via orthographic heuristics on Armenian upper-case Unicode ranges. Known loanwords are suppressed through a curated exclusion list.

The graph expands breadth-first: each processed entry contributes newly discovered derived or compound forms extracted from the “Compound words” (Բաղադրյալ բառեր, *Baghadryal barer*) section.

D. Part-of-Speech and Lexical Attributes

Part-of-speech (POS) tags are extracted by first matching Armenian Wiktionary template markers, then by section header search, and finally by heuristic suffix-based inference when structured signals are unavailable. The first definition line is retained as a node attribute and truncated to 300 characters for consistency.

E. Morphological Analysis and Edge Extraction

Derivational relations are extracted using a rule-based morphological analyzer implemented as a sequence of prioritized transformation rules grounded in Dum-Tragut [7]. For each lemma, we apply ordered rules corresponding to MEA’s major derivational processes, from most specific (e.g., reduplication, valency-changing morphology) to most general (e.g., reduplication, valency-changing morphology). Each successful match produces a directed edge from the base form to the derived one.

The system models several classes of derivation, including suffixation, prefixation, compounding, and valency-changing processes (e.g., causativization, detransitivization). All edges are typed: suffixation rules distinguish between denominal, deverbal, and deadjectival processes; prefixation rules are grouped into semantic classes (e.g., negation, intensification, directionality); compounding is represented as a multi-source relation linking the constituent roots to the compound form. This matters because nominalization and verbalization, though both suffixation processes, carry different syntactic implications. (See the Appendix A for definitions of linguistic terms.)

To account for regular stem alternations in Armenian (e.g., vowel deletion or root truncation), the analyzer incorporates a curated set of root-transformation patterns, enabling derived forms to be matched to their canonical bases. When no rule-based derivation can be identified, a fallback mechanism extracts component lemmas from the Wiktionary etymology section and reapplies the morphological analyzer.

F. Extraction Quality Validation

We assess extraction quality through coverage metrics, given the scale of the graph and the regularity of the grammar-grounded extraction rules. The graph contains 88,250 nodes and 174,414 edges (114,295 forward, 60,119

inverse). The most critical quality indicator is compound root detection: out of 54,910 compound edges, both constituent roots are successfully identified in 52,060 cases (94.8%). Edges where only the first root is found account for 4.3%, only the second root for 0.8%, and neither root for 0.1%. The gradient signal is dominated by well-attested relation types, limiting the effect of sparse noisy edges.

In addition, we estimated precision by drawing a random sample of 300 edges, which was then manually verified by an Armenian linguist. Of these, 266 were confirmed as valid derivational relations, yielding an estimated precision of 88.6%. Heuristic POS inference may introduce noise in the remaining cases, but graph learning remains robust due to large-scale structure and the dominance of high-frequency, well-attested relation types in the gradient signal.

G. Node Features

Each node is associated with a set of linguistically motivated features capturing core aspects of Armenian morphology. These include animacy, declension class, verbal transitivity, and aktionsart. Feature values are inferred using rule-based heuristics grounded in standard grammatical descriptions of MEA. When no inference is possible, the attribute is left undefined.

For use in GNNs, categorical features are mapped to learned embedding vectors, with an explicit category for unseen values. In addition, each node receives a character-level representation derived from the word’s surface form. Specifically, lemmas are padded, split into overlapping character trigrams, and processed by a Compact Convolutional Neural Network (CharCNN) [24].

H. Post-Processing and Graph Densification

Upon completion of the primary crawl, a reanalysis pass targets unresolved base forms, i.e., nodes that appear as edge sources but were not successfully retrieved or analyzed in the first iteration. By reapplying the morphological analyzer with the now-complete POS map, this step recovers links missed when the base word’s POS was not yet known at the time of first processing. The final datasets are then deduplicated and filtered to remove proper nouns, bare morphemes, and invalid entries shorter than two characters. The resulting graph is described in detail in Section V-A and Appendix B.

IV. MODEL ARCHITECTURE AND TRAINING

A. Input Feature Representation

All GNN models receive the same input feature matrix. Categorical node attributes are mapped to learned embedding vectors, with an explicit class for values absent from the training vocabulary. The word surface form is encoded by a compact CharCNN that processes character trigrams from the padded form, using convolutional kernels of width 3 and 5 to capture local subword patterns. The full input feature vector is the concatenation of the categorical embedding vectors and the CharCNN output.

B. Graph Convolutional Network

GCN [18] propagates information by averaging the features of each node’s immediate neighbors and then applying a learnable linear transformation, repeating this over two layers. GCN’s efficiency comes at a cost: treating all edges identically means a nominalization and a prefix negation contribute equally to a node’s representation, and since the model is transductive, it cannot embed new nodes without retraining. We add a skip connection concatenating the input features with the aggregated representation, preserving surface-form signal across layers.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{R})$ be the derivational graph, where $\mathcal{V}$ is the node set (lemmas), $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ is the typed edge set, and $\mathcal{R}$ is the set of 30 relation types. Each node $v \in \mathcal{V}$ is associated with an input feature vector $\mathbf{h}_v^{(0)} \in \mathbb{R}^d$. $\mathbf{H}^{(\ell)} \in \mathbb{R}^{|\mathcal{V}| \times d}$ denotes the node feature matrix at layer ℓ , where row v is $\mathbf{h}_v^{(\ell)}$.

We define $\tilde{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ as the symmetrically normalized adjacency matrix with added self-loops, where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}$ is its degree matrix. Let $\mathbf{W}^{(\ell)}$ denote a learnable weight matrix at layer ℓ , and σ denote the ReLU activation function. The GCN update rule [18] is then formulated as:

$$\mathbf{H}^{(\ell+1)} = \sigma(\tilde{\mathbf{A}} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)}). \quad (1)$$

C. GraphSAGE

GraphSAGE [19] extends the basic GNN framework with two modifications. First, it samples a fixed-size neighborhood at each layer rather than using the full neighborhood, which controls computational cost for high-degree nodes. Second, it learns a separate aggregation function (in our implementation, mean aggregation) rather than relying on the full normalized adjacency. Mean aggregation normalizes neighborhood representations by degree, which is appropriate given the highly skewed out-degree distribution in our graph: a small number of productive roots have several hundred descendants, and treating each neighbor equally prevents hub nodes from dominating the representations of their successors. GraphSAGE is inductive: new nodes can be embedded at inference time by running the learned aggregator over their neighbors. This matters for a living resource like Wiktionary.

The inductive aggregation rule [19] is:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \left[\mathbf{h}_v^{(\ell)} \parallel \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell)} \right]\right), \quad (2)$$

where $\mathcal{N}(v)$ is the sampled neighborhood of v and $[\cdot \parallel \cdot]$ denotes concatenation.

D. Graph Attention Network

GAT [20] replaces the fixed neighborhood average with a learned attention mechanism: each neighbor’s contribution is weighted by an attention coefficient computed from the features of both the query node and its neighbor. We

use GATv2 [21], which reorders the linear transformation and the attention nonlinearity to produce dynamic rather than static attention. In morphological graphs, identical suffixes often appear in both nominal and verbal contexts. Vanilla GAT fails to distinguish these distinct derivational roles due to its static attention mechanism. By computing attention after feature interaction, GATv2 assigns differentiated weights even to morphologically similar neighbors. Two attention heads are used; their outputs are concatenated and projected to the final embedding.

The dynamic attention coefficient [21] for edge (u, v) is:

$$e_{uv} = \mathbf{a}^\top \sigma \left(\mathbf{W} \left[\mathbf{h}_u^{(\ell)} \parallel \mathbf{h}_v^{(\ell)} \right] \right), \quad \alpha_{uv} = \frac{\exp(e_{uv})}{\sum_{k \in \mathcal{N}(v)} \exp(e_{kv})}, \quad (3)$$

with the node update $\mathbf{h}_v^{(\ell+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} \mathbf{W}' \mathbf{h}_u^{(\ell)} \right)$.

E. Relational Graph Convolutional Network

Unlike GCN, GraphSAGE, and GAT, R-GCN [4] conditions message passing on edge type. For a graph with 30 distinct relation types, this is the natural fit. For each edge type, a separate weight matrix transforms incoming messages before aggregation; a self-loop matrix handles the node's own features. With 30 distinct relation types in our graph, maintaining fully independent weight matrices would be parameter inefficient. R-GCN addresses this through basis decomposition: each relation-specific matrix is expressed as a learned linear combination of a shared set of basis matrices, reducing the parameter count while preserving the ability to discriminate between relation types. The cost of relational expressivity is slower training: R-GCN requires approximately four times longer per epoch than GCN due to the basis decomposition overhead. The auxiliary POS loss pushes embeddings toward POS-coherent clusters, and complements the link prediction objective.

The relational update rule [4] is:

$$\mathbf{h}_v^{(\ell+1)} = \sigma \left(\mathbf{W}_0^{(\ell)} \mathbf{h}_v^{(\ell)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{|\mathcal{N}_r(v)|} \mathbf{W}_r^{(\ell)} \mathbf{h}_u^{(\ell)} \right), \quad (4)$$

where $\mathcal{N}_r(v)$ denotes neighbors connected to v via relation r , and each $\mathbf{W}_r^{(\ell)}$ is decomposed via basis functions: $\mathbf{W}_r = \sum_{b=1}^B a_{rb} \mathbf{V}_b$, with shared bases $\{\mathbf{V}_b\}$ and relation-specific coefficients $\{a_{rb}\}$.

F. Distributional Baselines

Three distributional baselines are included to establish what can be learned without explicit graph structure or structural training objectives. Node2Vec [22] uses biased random walks over the graph structure followed by skip-gram training to learn positional embeddings, capturing co-occurrence in random-walk context windows but with no access to node features. FastText [9] trains character n-gram embeddings on random-walk sequences generated from the same graph using the DeepWalk procedure [25],

combining structural traversal order with surface-form signal. XLM-R [10] is a purely contextual baseline: its embeddings for Armenian lemmas are extracted by encoding each lemma as a single token and projecting the resulting 768-dimensional representation to 256 dimensions via PCA (keeping 96.7% of variance), with no fine-tuning and no access to graph structure. XLM-R tells us how far pretraining alone gets on Armenian morphology, without any graph or morphological supervision.

G. Ensemble

We test whether combining graph-derived and distributional embeddings improves over either alone. The best-performing GNN is identified by MRR on the test set, and paired with the distributional baseline that produces the highest ARI on held-out clustering. Their embeddings are fused through a gated mechanism, selected over concatenation, weighted interpolation, and simple summation based on an ablation study on seed 42, consistent with results from preliminary experiments across all five seeds. In the gated approach, a learned gate value is computed for each node from the concatenation of both embeddings, and the final representation is a convex combination of the two: the gate allows the model to rely more on the GNN for nodes embedded in dense derivational subgraphs, and more on the distributional signal for nodes at the periphery of the graph. The ensemble is decoded by the same link prediction decoder used for individual GNN training. The exact composition and fusion mode are reported in Section V-D.

The gate g_v is a d -dimensional vector rather than a scalar, making the fusion per-dimension rather than a single global interpolation weight. Formally, for GNN embedding $\mathbf{z}_v^{\text{gnn}}$ and distributional embedding $\mathbf{z}_v^{\text{aux}}$:

$$g_v = \sigma \left(\mathbf{W}_g \left[\mathbf{z}_v^{\text{gnn}} \parallel \mathbf{z}_v^{\text{aux}} \right] \right) \quad (5)$$

$$\mathbf{z}_v = g_v \odot \mathbf{W}_{\text{gnn}} \mathbf{z}_v^{\text{gnn}} + (1 - g_v) \odot \mathbf{W}_{\text{aux}} \mathbf{z}_v^{\text{aux}} \quad (6)$$

where $\mathbf{W}_g \in \mathbb{R}^{d \times (d_{\text{gnn}} + d_{\text{aux}})}$ is the gate projection, $\mathbf{W}_{\text{gnn}} \in \mathbb{R}^{d \times d_{\text{gnn}}}$ and $\mathbf{W}_{\text{aux}} \in \mathbb{R}^{d \times d_{\text{aux}}}$ project both embeddings into a shared space, σ is sigmoid, and $\odot$ denotes element-wise multiplication. The fused embedding $\mathbf{z}_v \in \mathbb{R}^d$ is passed to the asymmetric decoder.

H. Link Prediction Training

All GNN models are trained under the GAE [17] using an asymmetric decoder: $s(u, v) = (\mathbf{W}_{\text{src}} \mathbf{z}_u)^\top (\mathbf{W}_{\text{tgt}} \mathbf{z}_v)$, where $\mathbf{W}_{\text{src}}$ and $\mathbf{W}_{\text{tgt}}$ are learned linear projections. This makes the score directional ($s(u, v) \neq s(v, u)$), respecting the directed nature of derivational relations. The decoder is trained with pairwise ranking loss (Bayesian Personalized Ranking loss) [26] using hard negative samples that share the same POS as the positive tail.

Edges are split into train (80%), validation (10%), and test (10%) sets. At each training step, the encoder produces node embeddings; positive edge scores are computed

using the asymmetric decoder; and an equal number of hard negative samples are scored identically. Following standard practice in knowledge graph embedding, hard negatives are used during training to accelerate convergence and improve margin separation, while all ranking metrics are computed against uniformly sampled negatives to enable fair cross-model comparison [27].

The Adam optimizer is used with model-specific learning rates and weight decay values (Table I). The best checkpoint based on validation AUC-ROC is restored for the final test evaluation.

Under the closed-world assumption [28], any absent word pair is treated as a negative. Since the graph is incomplete, some negatives may be genuine derivational pairs; our metrics are therefore conservative estimates.

I. Implementation Details

All models were implemented in PyTorch Geometric and trained on a single CPU (32 GB RAM, 16 cores). Hyperparameters for all GNN models were tuned using Optuna; final values are given in Table I. Training time per model (single seed) was approximately: GCN ~18 min, GraphSAGE ~17 min, GAT ~19 min, R-GCN ~70 min (roughly four times slower than GCN due to basis decomposition overhead), Ensemble ~21 min. All experiments were run across five random seeds (42, 9, 100, 666, 4), controlling initialization, edge-split assignment, and negative sampling. Results are reported as mean $\pm$ standard deviation across seeds, with 95% bootstrap confidence intervals.

V. RESULTS

A. Dataset Scale and Composition

The construction pipeline produced 66,677 unique lemma nodes and 150,801 unique typed derivational edges after deduplication and cleaning. These figures show the graph used for model training and evaluation. Nouns are the most common POS (33,738 nodes), followed by adjectives (18,392), verbs (10,816), and adverbs (1,545), consistent with Armenian nominal suffixation being the most productive derivational process [6]. Causative and detransitive pairs are rarer but cover the full set of phonologically productive alternations. A subset of nodes carries a POS label “unknown” when neither template-based nor suffix-based inference succeeds; these are retained as they participate in valid morphological edges. Detailed information is presented in Appendix B.

B. Topological Properties

The graph is predominantly directed and acyclic, with low density, which is expected for derivational morphology, where a root spawns branches rather than forming dense clusters. The largest weakly connected component covers the majority of nodes. The out-degree distribution is long-tailed: a small number of highly productive roots (ինքն, *inqn*, “self”; քար, *qar*, “stone”; համ, *ham*, “taste”)

generate many derived nodes, while most nodes have few outgoing edges. Figure 1 illustrates this phenomenon for the most productive root, whose derivational family contains 417 derived words. The in-degree distribution is less extreme: most lemmas derive from only one or a few bases, and very few words have many incoming relations. More information regarding the graph topology can be found in Table VI.

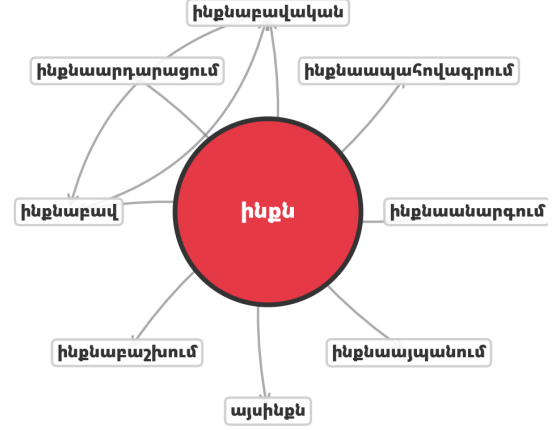


Fig. 1. Derivational family of ինքն. Each node is a word derived from this root; arrows indicate derivational relationships. The colored circle marks the root itself. The child node count is truncated to the first eight.

Automatic extraction can introduce occasional false positives. These are rare and do not qualitatively alter the observed patterns. For example, node ար (*ar*, “are”) in the family of համ (Figure 2).

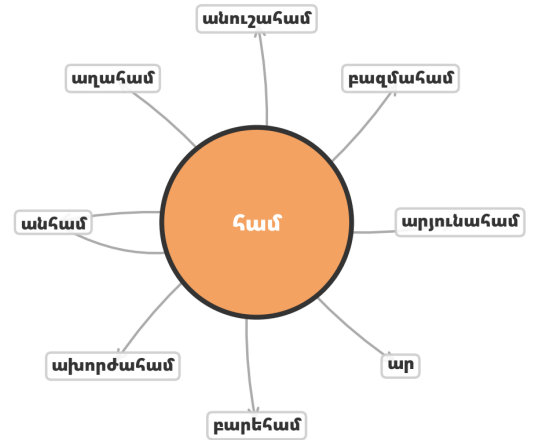


Fig. 2. Derivational family of համ. The child node count is truncated to the first eight.

TABLE I
FINAL HYPERPARAMETERS PER MODEL.

Model	embed_dim	hidden_dim	epochs	lr	dropout	weight_decay	extras
GCN	128	256	400	1.23e-3	0.2	3.48e-4	–
GraphSAGE	64	128	300	1.07e-3	0.3	6.16e-5	aggr = mean
GAT	64	128	200	6.20e-4	0.2	3.93e-4	heads = 2
R-GCN	64	256	200	8.59e-4	0.5	4.86e-4	num_bases = 32
Ensemble*	64	–	100	1e-3	–	–	gated fusion

*Not tuned with Optuna; set heuristically.

C. Link Prediction

Table II reports link prediction performance across all models, averaged over five seeds. Among individual GNNs, GCN achieves the highest AUC (0.9149) and MRR (0.4272), with Hits@10 of 0.6146, comfortably exceeding the RQ3 threshold. GAT achieves an AUC of 0.8772 and an MRR of 0.3219, while GraphSAGE achieves an AUC of 0.8821 and an MRR of 0.2973. R-GCN posts the weakest link prediction results among GNNs: AUC 0.8566, MRR 0.1724 (below the 0.20 threshold), and Hits@10 of 0.352. The gated ensemble (GCN and FastText) dramatically outperforms all individual models, achieving AUC 0.9983, MRR 0.9392, and Hits@10 0.9971.

The ensemble’s AUC (0.998 ± 0.0002) is unexpectedly high. Three structural factors are the most likely contributors, in decreasing order of certainty.

Armenian derivational morphology is phonologically systematic enough that a well-formed pair is nearly always distinguishable from a random negative by surface form alone. The sparsity of the graph ($\rho = 3.39 \times 10^{-5}$) compounds this: random node pairs are overwhelmingly non-derivational, making negatives trivially easy. Additionally, FastText was trained on random walks from the training graph, so character n-grams of test-set neighbors indirectly encode graph proximity. None of these constitute label leakage, but together they inflate AUC relative to a harder evaluation.

D. Ensemble Fusion Ablation

Table III reports results for all fusion modes. Gated fusion achieves the best AUC (0.9984) and MRR (0.9447) on seed 42. Concatenation (AUC 0.9983) and simple summation (0.9968) are competitive on AUC but weaker on MRR, while weighted interpolation (0.9981) ranks third. The gated mechanism’s advantage lies in its input dependence: for a node that is densely connected within the derivational graph, the gate can assign a high weight to the GNN embedding; for a peripheral node with fewer neighbors, it can rely more on the distributional signal. The GNN-only ablation (0.9234) and FastText-only ablation (0.9961) show that both components contribute, though FastText alone accounts for most of the gain.

E. Grammatical Category Clustering

GraphSAGE achieves the highest ARI among individual GNNs (0.2250), followed by R-GCN (0.2099) and GCN (0.1433). GAT yields a substantially lower ARI (0.0411), despite comparable link prediction performance. Among distributional baselines, FastText (0.0869) outperforms XLM-R (0.0268) and Node2Vec (0.0044). The ensemble’s ARI (0.0383) is lower than all individual GNNs, consistent with gated fusion being optimized for link prediction rather than clustering coherence. GraphSAGE’s Normalized Mutual Information (NMI) of 0.4837 is the highest among all models, reinforcing the ARI result.

F. Derivational Analogy Tests

Table IX reports accuracy at rank 1 for a representative selection of relation types across all models. The full per-relation breakdown over all 30 types is provided in Appendix C.

FastText achieves high accuracy on phonologically regular processes, such as detransitivization (0.988), reduplication (1.000), causativization (0.926), because the surface form alone identifies these transformations. FastText’s character n-gram similarity captures the pattern without needing any explicit graph signal. Node2Vec performs surprisingly well on several relation types (adjectivalization $\text{acc@1} = 0.760$, deadjectivalization = 0.828), suggesting that random-walk co-occurrence encodes some structural morphological signal even without node features. XLM-R stays near floor across all types (maximum $\text{acc@1} = 0.138$), showing that contextual pretraining offers little derivational signal for Armenian.

GCN achieves moderate accuracy on the most frequent relation types, with adjectivalization at $\text{acc@1} = 0.176$ and causativization at 0.166. GraphSAGE and R-GCN perform poorly on analogy tests despite strong clustering (GraphSAGE) and adequate link prediction (R-GCN), indicating that their embeddings organize derivational families coherently at a coarse level without encoding the precise directional vector offsets required by the analogy task. The ensemble substantially recovers analogy performance: detransitivization reaches $\text{acc@1} = 0.918$, reduplication = 0.928, negation prefixation = 0.754, and intensifying prefixation = 0.894. Compounding is the hardest category

TABLE II
LINK PREDICTION AND EMBEDDING QUALITY RESULTS (MEAN OVER 5 SEEDS $\pm$ STANDARD DEVIATION). EM-DASH (—) INDICATES THE METRIC WAS NOT EVALUATED FOR THAT MODEL.

Model	AUC	AP	F1	MRR	H@1	H@5	H@10	ARI	NMI
Ensemble	0.9983 $\pm$ 0.0002	0.9979 $\pm$ 0.0004	0.9844 $\pm$ 0.0013	0.9392 $\pm$ 0.0047	0.8938 $\pm$ 0.0081	0.9928 $\pm$ 0.0003	0.9971 $\pm$ 0.0002	0.0383 $\pm$ 0.0097	0.0915 $\pm$ 0.0177
GCN	0.9149 $\pm$ 0.0077	0.9172 $\pm$ 0.0100	0.8375 $\pm$ 0.0078	0.4272 $\pm$ 0.0441	0.3274 $\pm$ 0.0518	0.5328 $\pm$ 0.0347	0.6146 $\pm$ 0.0257	0.1433 $\pm$ 0.0121	0.2619 $\pm$ 0.0136
GraphSAGE	0.8821 $\pm$ 0.0042	0.8933 $\pm$ 0.0067	0.8011 $\pm$ 0.0053	0.2973 $\pm$ 0.0288	0.1892 $\pm$ 0.0262	0.4103 $\pm$ 0.0402	0.5214 $\pm$ 0.0306	0.2250 $\pm$ 0.0408	0.4256 $\pm$ 0.0731
GAT	0.8772 $\pm$ 0.0083	0.8862 $\pm$ 0.0094	0.7955 $\pm$ 0.0048	0.3219 $\pm$ 0.0182	0.2157 $\pm$ 0.0200	0.4342 $\pm$ 0.0198	0.5323 $\pm$ 0.0155	0.0411 $\pm$ 0.0197	0.0775 $\pm$ 0.0283
R-GCN	0.8566 $\pm$ 0.0032	0.8629 $\pm$ 0.0048	0.7835 $\pm$ 0.0035	0.1724 $\pm$ 0.0038	0.0836 $\pm$ 0.0039	0.2505 $\pm$ 0.0044	0.3520 $\pm$ 0.0069	0.2099 $\pm$ 0.0284	0.3667 $\pm$ 0.0348
FastText	—	—	—	—	—	—	—	0.0869	0.2073
Node2Vec	—	—	—	—	—	—	—	0.0044	0.0227
XLM-R	—	—	—	—	—	—	—	0.0268	0.0676

TABLE III
ENSEMBLE FUSION ABLATION RESULTS. GATE IS SELECTED AS THE BEST FUSION MODE.

Fusion mode	AUC	MRR	Hits@1	Hits@10
Gate (selected)	0.9984	0.9447	0.9031	0.9970
Concat	0.9983	0.9414	0.8963	0.9979
Weighted	0.9981	0.9298	0.8767	0.9976
Sum	0.9968	0.9416	0.8961	0.9985
GNN only	0.9234	0.4735	0.3856	0.6394
FastText only	0.9961	0.9401	0.8937	0.9974

across all models; the surface-form overlap between constituents and compound is unpredictable.

G. Nearest-Neighbor Qualitative Analysis

Table V reports the three nearest embedding-space neighbors for five seed words: **գիր** (*gir*, “writing”), **գրիչ** (*grich*, “pen”), **սիրել** (*sirel*, “to love”), **լույս** (*luys*, “light”), and **լուսավոր** (*lusavor*, “bright”).

The ensemble retrieves derivationally coherent neighbors across all five words. For **գիր**, all three neighbors are everyday document-type compounds on the same root (“diary”, “bulletin”, “bill of lading”). For **լուսավոր**, the neighbors are direct derivations: “illuminator”, “enlightened”, including **լուսավորյալ**, which FastText misses. This pattern holds across the other seed words: the ensemble consistently recovers morphologically precise neighbors rather than merely surface-similar ones.

FastText performs well on morphologically transparent words. For **գիր** and **լույս** it retrieves root-coherent compounds, and for **լուսավոր** it correctly identifies direct derivations. Its weakness is that character n-gram similarity cannot distinguish genuine morphological relatedness from accidental surface overlap: for **սիրել**, it retrieves **էլոքսիրել** (“to eloxate”), which shares the **-սիրել** string but has no derivational connection. For **գրիչ**, it recovers

the **-իչ** suffix pattern (agentive instruments) without root coherence; **տպագրիչ** and **գրգրիչ** share the suffix but not the writing root.

GCN’s results are mixed. For **գիր** it retrieves valid **-գիր** compounds, though more formal and obscure than the ensemble’s. For **գրիչ**, one neighbor (**հեռագրաստուն**, “telegraph office”) shares the writing root distantly, but the others – a medical term for diarrhea and a physics term for thermal current – are noise. For **լուսավոր**, GCN retrieves “emerald-adorned” and “felt-covered”: morphologically it is pattern-matching on the compound adjective structure (**-գարդ**, **-պատ** suffixes) rather than the **լու** root. Neighborhood averaging places nodes at the same graph distance in similar regions regardless of the specific derivational path, which explains both the partial successes and the scattered failures.

GraphSAGE produces the weakest individual-word results despite its strong clustering ARI. Its neighbors are consistently POS-coherent: nouns neighbor nouns, verbs neighbor verbs, but root identity is lost entirely. For **լույս** it retrieves “interest rate”, “opus”, and “curve”; for **լուսավոր**, “ribbon-bearing”, “mangy”, and “vaccine-related”. The mean aggregator normalizes degree effects across the full vocabulary, producing grammatically pure but morphologically uninformative neighborhoods.

TABLE IV
DERIVATIONAL ANALOGY TEST ACC@1 FOR SELECTED RELATION TYPES (SEED 42, $n = 300$ PER TYPE). FULL TABLE IN APPENDIX C.

Relation type	Node2Vec	FastText	XLM-R	GCN	SAGE	GAT	R-GCN	Ensemble
Nominalization	0.412	0.402	0.060	0.094	0.004	0.112	0.004	0.226
Adjectivalization	0.760	0.932	0.080	0.176	0.008	0.112	0.004	0.526
Verbalization	0.464	0.708	0.090	0.116	0.020	0.126	0.004	0.554
Negation prefix	0.248	0.680	0.002	0.040	0.004	0.024	0.014	0.754
Intensifying prefix	0.248	0.882	0.018	0.024	0.006	0.020	0.004	0.894
Causative	0.216	0.926	0.114	0.166	0.006	0.126	0.004	0.630
Detransitive	0.208	0.988	0.118	0.104	0.006	0.098	0.002	0.918
Diminutive	0.414	0.576	0.088	0.038	0.002	0.040	0.004	0.556
Reduplication	0.106	1.000	0.008	0.146	0.008	0.012	0.008	0.928
Noun compounding	0.402	0.322	0.020	0.018	0.008	0.026	0.006	0.428
Deadjectivalization	0.828	0.890	0.066	0.168	0.010	0.126	0.002	0.394

XLM-R’s failures are qualitatively different. For զիւր it retrieves radio-telegraphy equipment and signal transmission terms – semantically adjacent through the communication domain but morphologically unrelated. For լույս it retrieves the տեսաթոմբ (“thalamus”), presumably through scientific co-occurrence in the pretraining corpus. For լուսավոր it retrieves ժամատուն (chapel), likely through religious text co-occurrence with the Armenian word for enlightenment. These results confirm that contextual pretraining encodes topical association rather than derivational structure.

VI. ANALYSIS AND VALIDATION

A. RQ1: Dataset Coverage and Graph Properties

The construction pipeline yields a derivational morphology graph with 66,677 nodes and 150,801 typed edges can be constructed from the Armenian Wiktionary at the scale needed for learning. The long-tailed out-degree distribution (mean 2.26, maximum 417) matches the known productivity hierarchy of Armenian roots documented in Galstyan [6]: a small set of roots is highly generative while the majority contribute a single derivational branch. The 30 relation types map onto the morphological process classes in Dum-Tragut’s taxonomy [7], providing external linguistic validation. The reanalysis pass recovered 3,872 edges missed during the first crawl due to initially unresolved POS labels on base words, confirming that iterative construction is necessary for reasonable coverage.

B. RQ2: Embedding Quality

GNN architectures substantially outperform distributional baselines on grammatical category clustering, addressing RQ2. GraphSAGE (ARI = 0.225) and R-GCN (0.210) are the strongest individual models, both exceeding FastText (0.087), XLM-R (0.027), and Node2Vec (0.004). The ARI gap between GNNs and FastText is what graph structure contributes, on top of whatever character-level signal FastText already provides. Message passing

propagates a morphological family signal that distributional methods cannot fully recover because surface-form co-occurrence does not distinguish the typed relational structure of edges.

The result for GraphSAGE is particularly notable: despite weaker link prediction MRR than GCN, it produces more POS-coherent clusters. Mean aggregation appears to encourage embeddings that reflect grammatical category regularities across the full vocabulary distribution, without being dominated by high-degree hub nodes. The nearest-neighbor analysis (Table V) bears this out: GraphSAGE’s neighbors are always the right POS, even when the specific root is wrong.

Visual evidence from t-SNE projections (Figure 3 and Figure 4) supports this quantitative finding. GraphSAGE embeddings separate parts of speech into distinct, compact regions, whereas FastText produces more diffuse and overlapping clusters.

C. RQ3: Link Prediction

All four GNN models, except R-GCN, exceed the Hits@10 ≥ 0.35 threshold (GCN: 0.615, GAT: 0.532, GraphSAGE: 0.521, R-GCN: 0.352) and the MRR ≥ 0.20 threshold (GCN: 0.427, GAT: 0.322, GraphSAGE: 0.297).

R-GCN’s underperformance (MRR of 0.172) relative to GCN is the most notable finding in the link prediction results. Despite being the only model designed to condition aggregation on edge type, it records the lowest MRR and AUC among GNNs. The most likely explanation is data sparsity per relation type. With 30 relations, several have fewer than 100 edges. For these sparse relations, the basis matrices cannot be estimated reliably even with basis decomposition, and the model effectively overfits relation-specific components while the well-represented types (noun compounding, general suffixation) dominate the gradient signal. GCN’s homogeneous message passing, by treating all edges equally, yields a more stable gradient across the entire edge set. This finding aligns with

TABLE V
THREE NEAREST EMBEDDING-SPACE NEIGHBORS FOR FIVE SEED WORDS (SEED 42).

Seed word	FastText	XLM-R	GCN	GraphSAGE	Ensemble
գիր	բալլադագիր, օրագիր, եկագիր	ռադիոհեռուիտակ, ռադիոհեռագիր, ձայնահաղորդականություն	վիճակագիր, նամակագիր, կարգագիր	ճոռոմախոսություն, սիրող, սյունակել	օրագիր, ազդագիր, բաժագիր
գրիչ	տպագրիչ, ցրիչ, գրգռիչ	գրքույկ, մատյան, դակիչ	հեռագրատուն, հարինք, ջերմահոսանք	ցուցմունք, սրահ, գունդ	տպագրիչ, գրգռիչ, գրիպ
սիրել	էրոսսիրել, սիրելի, իմաստասիրել	վստահ, հետաքրքրություն, ստորադրել	համաձայնել, հաճոյախոսել, հավաստիանալ	մարգել, ծովացնել, սապոնացնել	էրոսսիրել, իմաստասիրել, սիրվել
լույս	լուսելույս, շվաքլույս, լուսնելույս	լիս, լուսաբեկում, տեսաթումբ	մացառ, ոլորահյուս, ոսկեքնար	տոկոս, երկ, կոր	լուսելույս, շվաքլույս, լուսնելույս
լուսավոր	լուսավորիչ, լուսավորչական, լուսավորել	ժամատուն, լուսանալ, լուսնաշուրջ	գմրուխտազարդ, թաղիքապատ, գմրուխտազարդել	ժապավենավոր, քոսոտ, պատվաստային	լուսավորիչ, լուսավորչական, լուսավորյալ

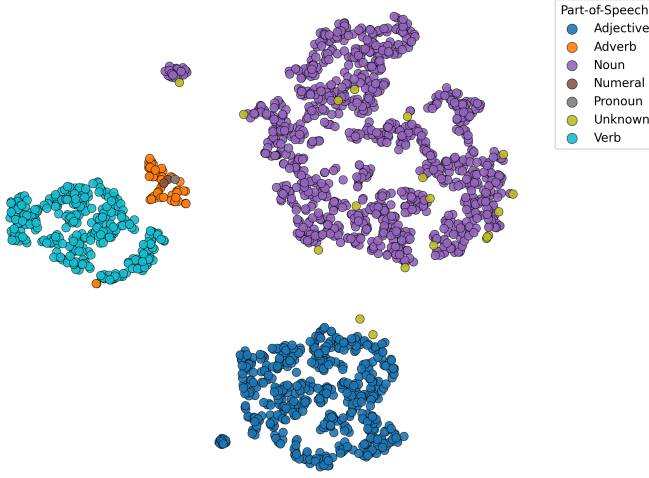


Fig. 3. t-SNE projection of GraphSAGE embeddings colored by POS (same 2000 words sampled proportionally from the common vocabulary). Each dot is a word; colors correspond to universal POS tags.

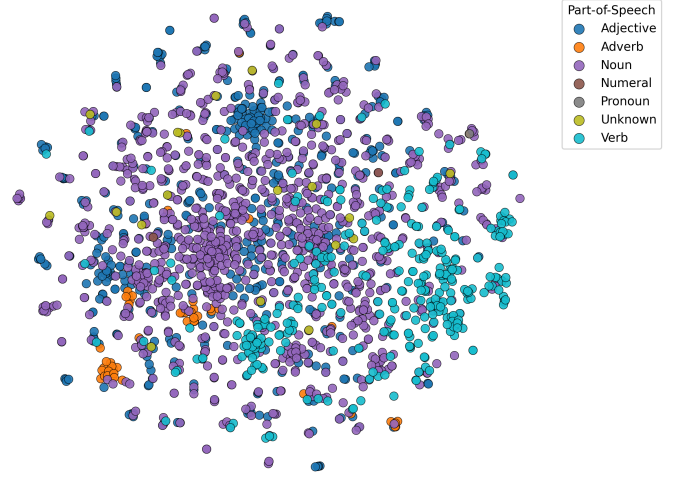


Fig. 4. t-SNE projection of FastText embeddings using the same 2000 words and color scheme as Figure 3.

observations in the knowledge graph embedding literature that relational models require a minimum frequency per relation type to outperform simpler baselines [28].

The analogy test results provide an additional lens on model differences. FastText’s near-perfect performance on surface-form-predictable relation types (detransitivization, reduplication, causativization) confirms that these processes are fully recoverable from character n-gram statistics without any graph signal. GCN recovers partial analogy structure for the most frequent relation types, but the moderate accuracy (adjectivalization $\text{acc@1} = 0.176$) indicates that neighborhood averaging blurs the precise directional offset required for analogy. The ensemble recovers analogy performance across both suffixation and prefixation types, with compounding remaining the most challenging category for all models.

D. Error Analysis

GraphSAGE was chosen to illustrate the failure modes of the link predictor. False negatives show morphological relations that missed clear derivational links due to low training frequency or directional ambiguity (Figure 5). False positives are non-relations that the model hallucinated with full confidence, typically when there was no actual derivational edge present (Figure 6).



Fig. 5. False negatives. From left to right: *dramap'okh*, “money changer”; *dramap'okhut'yun*, “currency exchange”; *yerkar*, “long”; *yerkaraknchit'*, “weevil.”



Fig. 6. False positives. From left to right: *batsvel*, “to open up”; *aynchap*, “so much”; *totalitar*, “totalitarian”; *druyt*, “thesis.”

E. Limitations

The POS and node feature labels are heuristically inferred and will contain errors, particularly for the 1,612 unknown-labeled nodes; integration with a dedicated Armenian morphological analyzer would improve label quality. Also, the dataset is a Wiktionary snapshot: derivational links present in Galstyan [6] but absent from Wiktionary are not captured, and coverage is uneven across frequency bands. The graph models derivational morphology exclusively and does not capture inflectional paradigms (case, number, tense), semantic polysemy, or contextual lexical usage. Homographs are the words that share orthographic form but differing in meaning or etymology, e.g., *սեր* (*ser*, “love” / “milk cream”). They are represented as a single node, and conflate distinct lexical entries. This limits the graph’s utility for downstream tasks requiring sense disambiguation.

VII. DISCUSSION AND CONCLUSION

A. Main Contributions

This work produces two outputs that directly addresses the resource gap for computational Armenian. The first is the dataset itself: the first large-scale, typed, machine-readable derivational morphology resource for MEA, built by combining community-sourced Wiktionary data with grammar-grounded morphological analysis anchored in Dum-Tragut [7]. The second is the evaluation showing that GNN embeddings outperform distributional baselines on clustering, and that link prediction exceeds the thresholds set in RQ3 for three of four architectures.

B. Key Findings

The construction pipeline demonstrates that a large-scale typed derivational graph for Armenian is buildable from Wiktionary when extraction rules are anchored in a reference grammar. The resulting graph’s topological properties – a long-tailed out-degree distribution, a dominant weakly connected component, and 30 relation types mapping onto Dum-Tragut’s morphological taxonomy – match what is known about Armenian derivational productivity, providing external validation that the pipeline is capturing genuine linguistic structure rather than extraction artifacts.

The evaluation reveals a clean dissociation between the two tasks. For link prediction, GCN outperforms all other individual architectures, including R-GCN, which is the

model architecturally designed for typed graphs. The most plausible explanation is distributional: with over 30 relation types and several represented by fewer than a hundred edges, R-GCN’s relation-specific parameters cannot be estimated reliably, and homogeneous aggregation turns out to be more stable under that sparsity. For grammatical category clustering, GraphSAGE leads, despite its moderate link prediction MRR. Its mean aggregator, by normalizing degree effects, produces POS-coherent embeddings across the full vocabulary without being pulled toward high-degree hub nodes. A model can organize derivational families by grammatical category without encoding the precise directional structure that link prediction requires, and these results show that different architectures optimize for each.

Gated ensemble fusion outperforms both components and all fusion ablations, with the gate learning to weight the GNN embedding more heavily for densely connected nodes and the distributional signal more heavily at the graph’s periphery.

C. Implications

The dataset is available for use in any downstream Armenian NLP task that could benefit from morphological family information, including morphological tagging, lemmatization, and low-resource machine translation. More broadly, the methodology of grammar-grounded Wiktionary scraping combined with GNN training is transferable to other MRLs.

D. Future Work

Coverage could be improved. Semi-automatic alignment with digitized portions of Galstyan’s dictionary would extend the graph beyond Wiktionary, particularly for rarer lexical items absent from community-sourced entries. Graph enrichment is another promising direction; adding etymological properties, full word families beyond direct derivations, and homograph distinctions would enable finer-grained representation learning and make the resource more useful for sense disambiguation tasks.

On the modeling side, R-GCN’s underperformance points to a problem worth solving: relation-frequency-weighted basis allocation or relational dropout may recover the theoretical advantage of relational message passing under sparse edge distributions. An ensemble pairing R-GCN with FastText is currently being trained toward this end. Additionally, the embeddings produced here have not yet been evaluated on downstream Armenian NLP tasks, such as morphological tagging, lemmatization, and low-resource machine translation.

REFERENCES

- [1] C. Arnett and B. K. Bergen, “Why do language models perform worse for morphologically complex languages?” 2024. [Online]. Available: <https://arxiv.org/abs/2411.14198>

- [2] E. Asgari, Y. E. Kheir, and M. A. S. Javaheri, “MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.00894>
- [3] H. Avetisyan, C. Karasavva, and D. Brioneske, “Morphcraft: Morphology-aware transformers for armenian and greek,” in *2025 20th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, 2025, pp. 1–6.
- [4] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1703.06103>
- [5] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” in *Proceedings of the 27th International Conference on Neural Information Processing Systems (NIPS)*, vol. 2, 2013, pp. 2787–2795.
- [6] S. Galstyan, *School Word-Formation Dictionary*. Yerevan: Zangak Publishing, 2014, in Armenian.
- [7] J. Dum-Tragut, *Armenian: Modern Eastern Armenian*, ser. London Oriental and African Language Library. John Benjamins Publishing Company, 2009, no. 14.
- [8] H. Acharyan, *Armenian Etymological Dictionary*. Yerevan: AAS Press, 1971, in Armenian.
- [9] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” 2016. [Online]. Available: <https://arxiv.org/abs/1607.04606>
- [10] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” 2020. [Online]. Available: <https://arxiv.org/abs/1911.02116>
- [11] L. Wu, Y. Chen, K. Shen, X. Guo, H. Gao, S. Li, J. Pei, and B. Long, “Graph neural networks for natural language processing: A survey,” 2022. [Online]. Available: <https://arxiv.org/abs/2106.06090>
- [12] R. Navigli and S. P. Ponzetto, “Babelnet: The automatic construction, evaluation and application of a wide-coverage multilingual semantic network,” *Artificial Intelligence*, vol. 193, pp. 217–250, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370212000793>
- [13] J. A. Botha and P. Blunsom, “Compositional morphology for word representations and language modelling,” in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014. [Online]. Available: <https://arxiv.org/abs/1405.4273>
- [14] Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” 2020. [Online]. Available: <https://arxiv.org/abs/2003.01332>
- [15] T. Zesch, C. Müller, and I. Gurevych, “Using wiktionary for computing semantic relatedness,” in *Proceedings of the National Conference on Artificial Intelligence (AAAI)*, vol. 2, 2008, pp. 861–866.
- [16] G. Sérasset, “Dbnary: Wiktionary as a lemon-based multilingual lexical resource in RDF,” *Semantic Web*, vol. 6, pp. 355–361, 2015.
- [17] T. N. Kipf and M. Welling, “Variational graph auto-encoders,” 2016. [Online]. Available: <https://arxiv.org/abs/1611.07308>
- [18] —, “Semi-supervised classification with graph convolutional networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [19] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” 2018. [Online]. Available: <https://arxiv.org/abs/1706.02216>
- [20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- [21] S. Brody, U. Alon, and E. Yahav, “How attentive are graph attention networks?” *arXiv preprint arXiv:2105.14491*, 2021. [Online]. Available: <https://arxiv.org/abs/2105.14491>
- [22] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1607.00653>
- [23] Vortan, “Hunspell dictionaries for eastern and western armenian,” <https://github.com/Vortan/dictionaries>, open-source project supporting Armenian NLP tools. Accessed: 2026-05-15.
- [24] X. Zhang, J. Zhao, and Y. LeCun, “Character-level convolutional networks for text classification,” 2016. [Online]. Available: <https://arxiv.org/abs/1509.01626>
- [25] B. Perozzi, R. Al-Rfou, and S. Skiena, “Deepwalk: Online learning of social representations,” in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701–710.
- [26] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “Bpr: Bayesian personalized ranking from implicit feedback,” 2012. [Online]. Available: <https://arxiv.org/abs/1205.2618>
- [27] B. Shi and T. Weninger, “Open-world knowledge graph completion,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.03438>
- [28] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich, “A review of relational machine learning for knowledge graphs,” *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2016.

APPENDIX A

GLOSSARY OF LINGUISTIC AND TECHNICAL TERMS

Linguistic Terms

Adjectivalization. Formation of adjectives from nouns or verbs. Inverse process – deadjectivalization.

Aktionsart. Lexical aspect that describes the internal temporal structure of events.

Animacy. Semantic feature that indicates whether a noun refers to a living entity.

Compounding. Formation of words by combining multiple lexical roots.

Derivational morphology. The process of creating new lexemes from existing ones via affixation or other transformations that change meaning or grammatical category.

Diminutive. A derivation that expresses smallness or affection.

Inflectional morphology. Grammatical modification that does not create new lexemes but encodes features such as tense, case, or number. Inflection is excluded from the graph.

Lemma. The canonical dictionary form of a word. All inflected forms map to a single node in the graph.

Morpheme. The smallest meaningful unit in a language. Roots carry core meaning, while prefixes and suffixes modify it.

Morphologically rich language (MRL). A language where words encode substantial grammatical information via morphology rather than syntax. Examples include Armenian, Turkish, Finnish, and Arabic.

Morphology. The branch of linguistics concerned with word structure – how smaller meaningful units combine to form words. It distinguishes inflectional and derivational processes.

Nominalization. Derivation of nouns from verbs or adjectives. Inverse process – denominalization.

Part-of-speech (POS). A grammatical category such as noun, verb, or adjective. Used both as a node attribute and for evaluation.

Reduplication. Morphological repetition used for intensity or emphasis.

Valency. The number of core arguments a verb selects (e.g., subject, object). Valency-changing operations such as causativization increase this number, while detransitivization decreases it. These changes are explicitly modeled as typed edges in the graph.

Verbalization. Formation of verbs from nouns or adjectives. Inverse process – deverbalization.

Technical Terms

accuracy@K (acc@K). Proportion of cases where the correct target word appears among the top-K nearest neighbors in the embedding space (used to evaluate derivational analogy tests).

Adjusted Rand Index (ARI). Clustering metric corrected for chance agreement.

AUC-ROC. Area Under the ROC Curve; measures binary classification quality independent of threshold.

Basis decomposition. Parameter-sharing technique in R-GCN that expresses relation weights as combinations of shared bases.

Concatenation fusion. Ensemble method that concatenates embeddings from different models and passes them through a linear layer.

Gated fusion. Adaptive ensemble method combining embeddings via learned per-node weights.

Graph Autoencoder (GAE). Encoder-decoder framework for reconstructing graph structure from embeddings.

Hits@K (H@K). Proportion of cases where the correct item appears in the top-K predictions (used for link prediction ranking).

Link prediction. Task of predicting whether an edge exists between two nodes.

Mean Reciprocal Rank (MRR). Ranking metric measuring how highly the correct item is ranked on average.

Normalized Mutual Information (NMI). Clustering metric quantifying the information overlap between predicted clusters and true labels, normalized to $[0, 1]$.

Summation fusion. Ensemble method that adds embeddings from different models element-wise.

Weighted interpolation fusion. Ensemble method that averages embeddings using fixed (non-learned) weights.

APPENDIX B DATASET STATISTICS

A. Raw vs. Post-Processing Node and Edge Counts

The raw files extracted from the Wiktionary scrape contain more rows (174,387) than the training graph because

TABLE VI
SUMMARY STATISTICS OF THE CONSTRUCTED MORPHOLOGICAL GRAPH.

Property	Value
<i>Graph size</i>	
Nodes	66,677
Edges	150,801
<i>Connectivity</i>	
Largest weakly connected component	51,274 (76.9%)
Weakly connected components	6,567
Strongly connected components	34,962
<i>Degree</i>	
Average in/out degree	2.26
Median in-degree	2
Median out-degree	1
Maximum in-degree	28
Maximum out-degree	417
<i>Miscellaneous</i>	
Graph density	3.39×10^{-5}

TABLE VII
POS DISTRIBUTION OF NODES.

POS	Nodes	% of total
Noun	33,738	50.60%
Adjective	18,392	27.58%
Verb	10,816	16.22%
Unknown	2,042	3.06%
Adverb	1,545	2.32%
Numeral	103	0.15%
Pronoun	25	0.04%
Preposition	9	0.01%
Conjunction	4	0.01%
Modal	3	0.00%

the post-processing pipeline applies deduplication, proper-noun filtering, bare-morpheme removal, and minimum-length filtering. Additionally, 23,185 isolated nodes (nodes with no edges after cleaning) are excluded before training. The Table VI reports the post-processing counts as used in all experiments. The POS distribution is given in Table VII.

B. Relation Type Distribution

Forward and inverse relation pairs are listed together. Edge counts reflect the post-processing graph. The details given in Table VIII.

APPENDIX C FULL DERIVATIONAL ANALOGY RESULTS

Table IX reports accuracy at rank 1 for all tested relation types, for all models. Forward and inverse relation types are separated to allow assessment of directional asymmetry in embedding structure.

FastText dominates on phonologically regular, surface-form-transparent processes: it reaches near-ceiling on reduplication (1.000), detransitivization (0.988), and detransitivization inverse (0.998), and performs strongly across all prefix types – negation (0.680), intensifying (0.882), locative (0.806), temporal (0.818) – where the transformation

TABLE VIII
DISTRIBUTION OF DERIVATIONAL RELATION TYPES IN THE
CONSTRUCTED GRAPH.

Relation type	Edges	% of total
General suffixation*	47,692	27.35%
Noun compounding	36,268	20.80%
Nominalization	17,104	9.81%
Denominalization	16,292	9.34%
Adjective compounding	12,189	6.99%
Deverbalization	9,097	5.22%
Synthetic compounding	6,378	3.66%
Verbalization	6,229	3.57%
Detransitivization*	4,956	2.84%
Adjectivalization	4,944	2.83%
Deadjectivalization	4,944	2.83%
Negation prefixation*	2,736	1.57%
Causativization*	1,882	1.08%
Diminutive suffixation	1,126	0.64%
Directional prefixation*	1,066	0.61%
Temporal prefixation*	580	0.33%
Locative prefixation*	414	0.24%
Intensifying prefixation*	280	0.16%
Demonym*	132	0.08%
Compound component	45	0.03%
Reduplication	33	0.02%

*forward and inverse directions

is a consistent string operation recoverable from character n-grams alone. The ensemble inherits this strength and extends it: it leads on most relation types where FastText already performs well, and recovers partial accuracy on types where FastText struggles, such as noun compounding (0.428) and compound component (0.088).

Individual GNNs are weakest on compounding. Noun compounding, synthetic compound, adjective compound, and compound component are near-floor for GraphSAGE, GAT, and R-GCN, and only marginally better for GCN. This is expected: compound formation in Armenian involves heterogeneous phonological patterns with no consistent surface signal, and the graph structure alone does not provide a reliable directional offset for the analogy task. Node2Vec performs surprisingly well on deadjectivalization (0.828) and adjectivalization (0.760), suggesting that random-walk co-occurrence captures some structural regularity for these relation types even without node features.

The forward-inverse asymmetry is inconsistent across relation types. For causativization, the forward direction is easier (FastText: 0.926 vs. 0.790 inverse); for detransitivization and most prefix types, the inverse is comparable or higher. XLM-R stays near floor across all 30 types, with causativization inverse its single best result (0.138), confirming that contextual pretraining provides no meaningful derivational signal for Armenian regardless of relation type or directionality.

TABLE IX
DERIVATIONAL ANALOGY ACC@1 ACROSS RELATION TYPES (SEED 42, $n = 300$ PER TYPE).

Relation type	Node2Vec	FastText	XLM-R	GCN	GraphSAGE	GAT	R-GCN	Ensemble
Nominalization	0.412	0.402	0.060	0.094	0.004	0.112	0.004	0.226
Adjectivalization	0.760	0.932	0.080	0.176	0.008	0.112	0.004	0.526
Verbalization	0.464	0.708	0.090	0.116	0.020	0.126	0.004	0.554
General suffixation	0.306	0.110	0.092	0.022	0.004	0.018	0.006	0.186
Demonym	0.324	0.768	0.070	0.048	0.006	0.008	0.002	0.666
Negation prefix	0.248	0.680	0.002	0.040	0.004	0.024	0.014	0.754
Intensifying prefix	0.248	0.882	0.018	0.024	0.006	0.020	0.004	0.894
Directional prefix	0.294	0.678	0.006	0.108	0.004	0.028	0.004	0.536
Locative prefix	0.352	0.806	0.018	0.032	0.008	0.018	0.006	0.722
Temporal prefix	0.224	0.818	0.008	0.068	0.006	0.026	0.002	0.586
Noun compounding	0.402	0.322	0.020	0.018	0.008	0.026	0.006	0.428
Synthetic compound	0.166	0.198	0.002	0.008	0.004	0.016	0.002	0.286
Adjective compound	0.164	0.078	0.014	0.032	0.002	0.014	0.004	0.270
Compound component	0.490	0.110	0.034	0.044	0.002	0.008	0.004	0.088
Causativization	0.216	0.926	0.114	0.166	0.006	0.126	0.004	0.630
Detransitivization	0.208	0.988	0.118	0.104	0.006	0.098	0.002	0.918
Diminutive	0.414	0.576	0.088	0.038	0.002	0.040	0.004	0.556
Reduplication	0.106	1.000	0.008	0.146	0.008	0.012	0.008	0.928
Denominalization	0.360	0.262	0.036	0.076	0.004	0.018	0.006	0.194
Deadjectivalization	0.828	0.890	0.066	0.168	0.010	0.126	0.002	0.394
Deverbalization	0.408	0.226	0.008	0.054	0.004	0.014	0.002	0.208
General suffixation inverse	0.296	0.114	0.004	0.044	0.002	0.010	0.004	0.070
Demonym inverse	0.482	0.690	0.030	0.044	0.004	0.012	0.002	0.582
Negation prefix inverse	0.346	0.788	0.002	0.060	0.004	0.020	0.004	0.822
Intensifying prefix inverse	0.258	0.648	0.014	0.064	0.004	0.016	0.004	0.474
Directional prefix inverse	0.594	0.622	0.018	0.046	0.004	0.014	0.004	0.428
Locative prefix inverse	0.342	0.786	0.018	0.040	0.004	0.014	0.004	0.392
Temporal prefix inverse	0.336	0.748	0.004	0.012	0.002	0.008	0.004	0.464
Causativization inverse	0.270	0.790	0.138	0.238	0.010	0.136	0.004	0.560
Detransitivization inverse	0.374	0.998	0.058	0.162	0.004	0.054	0.002	0.878